Proyecto

(Desarrollo: 3 sesiones, Competición: 1 sesión)

Introducción

Después de dominar cómo controlar un robot con cinemática diferencial para que no se choque mientras explora el entorno, desarrollar la habilidad de localizarse con respecto a un mapa conocido, aprender a planificar una ruta, y seguirla, ¡ha llegado la hora del último reto! Su misión, si decide aceptarla[1], es asegurarse de que todas las piezas funcionen de forma conjunta para que contraten a WALL·T y sus hermanos en los almacenes automatizados de Amazin' Technologies, una compañía multinacional de logística. El proceso de selección es duro, y todos los candidatos competirán para demostrar que merecen el puesto.

Objetivos

Al concluir el proyecto el alumno deberá ser capaz de:

- Explicar en detalle cómo funcionan los algoritmos básicos que permiten a un robot móvil operar de forma autónoma en entornos estáticos cuando se dispone de un mapa.
- Desarrollar y depurar *software* distribuido para robots móviles a través de la interconexión de nodos de ROS 2 utilizando el mecanismo de publicación/suscripción.
- Transferir algoritmos desarrollados en simulación a un robot físico y lidiar con las restricciones computacionales del *hardware* embarcado.

Evaluación

Cada grupo entregará a través de Moodle un **único archivo comprimido** —en formato `.zip` o `.7z`— que incluya los entornos de simulación (🗀 `sim_ws`) y del robot físico (🗀 `tb3_ws`) completos, así como una **presentación** donde se expliquen las mejoras realizadas sobre los módulos de las prácticas y los principales problemas a los que se ha tenido que enfrentar. La fecha límite son las **15:00 horas del día de la competición**. La calificación del proyecto se obtendrá atendiendo a los siguientes criterios:

- **Presentación (15 %):** Se prestará especial énfasis a la calidad del material de soporte, la comunicación oral y a la explicación de los cambios introducidos tanto en simulación como para portar el código al robot real.
- **Calidad del código (15 %):** Se tendrá en cuenta la claridad, la organización, la documentación, que se cumplan las convenciones de estilo habituales como PEP 8... Recuerde también que el código debe estar escrito íntegramente en inglés.
- **Operación en simulación (50 %):** Los profesores ejecutarán el código en 6 combinaciones distintas de puntos iniciales y finales, incluyendo los de la competición. Se probará cada combinación dos veces. Si en al menos uno de los intentos se alcanza el destino se sumarán dos puntos, y si el robot solo logra localizarse, uno. Se descartará el peor intento.

[1] Más le vale porque una parte no despreciable de la nota de la asignatura depende de ello.

- **Operación en la realidad (20 %):** Se evaluará durante la competición. Si el robot alcanza el objetivo en cualquiera de los dos intentos se sumará la puntuación completa. Si solo se localiza, la mitad.
- **Contribución:** La nota se puede multiplicar por un factor mayor o menor que uno para tener en cuenta el nivel de involucración de los diferentes miembros del equipo.

En la competición se podrán conseguir puntos extra que se añadirán a la nota sobre 10 puntos. Esto significa que potencialmente se pueden conseguir hasta **12 puntos**, los cuales se utilizarán directamente —sin saturar a 10— para calcular la media ponderada de la asignatura.

Preparación

Con lo que ha desarrollado en las prácticas hasta la fecha, ya dispone de casi todo lo que necesita para tener un robot móvil autónomo funcional. No obstante, para facilitarle un poco la tarea, en Moodle encontrará algunos archivos de código útiles.

- Para el espacio de trabajo de **simulación** (🗀 `sim_ws`) solo hay un archivo de lanzamiento `project.launch.py` que se encarga de arrancar todos los nodos que ha desarrollado durante las prácticas en el orden correcto. Siéntase libre de modificar todos los hiperparámetros a su gusto.
- Para el **robot físico** (🗀 `tb3_ws`), encontrará un nodo adicional `monitoring_node.py` que debe colocar en el paquete 🗀 `amr_turtlebot3` —no olvide añadir el punto de entrada en `setup.py`—. Restituye algunas funcionalidades perdidas al eliminar el nodo de CoppeliaSim. En concreto, se encarga de monitorizar y mostrar en el terminal las coordenadas en las que el robot se localiza por primera vez, así como el tiempo total que tarda en llegar a su destino. También se incluye otro archivo de lanzamiento `project.launch.py`, distinto del de simulación, que ejecuta el nodo de monitorización junto con todos los demás que ha desarrollado en las prácticas para el robot real. **Compruebe que el nombre del nodo de odometría coincide** con el que eligió en la Práctica 2 y modifique el archivo de lanzamiento en caso contrario.

Normas de la competición

La competición tendrá dos fases diferenciadas: una primera parte en **simulación**, y una segunda con el **robot físico**. En ambos casos las condiciones serán las mismas. Se situará el robot en una pose aleatoria del entorno mostrado en la Figura 1[2], se le pedirá que se localice y, posteriormente, intente llegar lo más rápido posible a una ubicación determinada. Las coordenadas inicial y final son alto secreto, y no serán desveladas hasta cinco minutos antes de la competición. ¡Prepare a su robot para lo inesperado! Lo que sí sabe, no obstante, es que el robot solo puede empezar orientado hacia uno de los cuatro puntos cardinales: norte, sur, este u oeste. No olvide contemplar la posibilidad de que comience mirando a una pared. Igual que en la Fórmula 1, existe parque cerrado, es decir, **no está permitido modificar los parámetros** durante la competición, de modo que asegúrese de entregar a través de Moodle el conjunto más robusto que encuentre.

Al entrar en el aula, tendrá que descargar en el ordenador de sobremesa de su puesto —para que todo el mundo compita en condiciones de igualdad de *hardware*— el código de su entrega de Moodle y se asegurará de que la simulación funciona correctamente. Una vez esté todo preparado, el profesor indicará los puntos inicial y final y se arrancarán las simulaciones de forma simultánea. Se ejecutará el

[2] En realidad, no es ninguna sorpresa. Ya sabía cómo era el entorno porque su agente encubierto Sonny consiguió hacerse con los planos durante la Práctica 4...

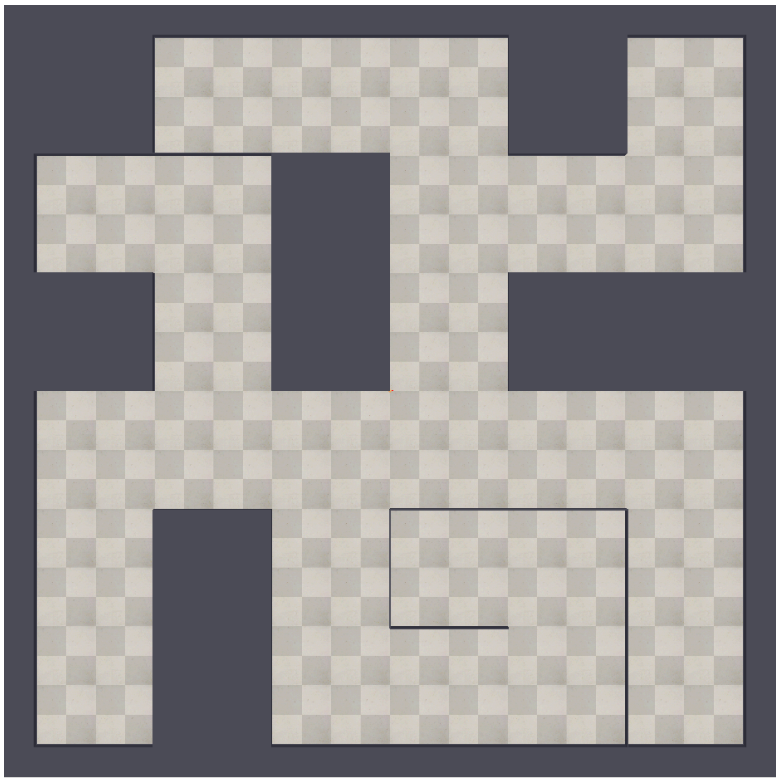

Figura 1. Entorno de tipo laberinto para la competición.

programa dos veces y solo se considerará el mejor tiempo de ejecución. Modificar las funciones que calculan el tiempo de ejecución en el nodo de CoppeliaSim o en el nodo de monitorización del robot físico está terminantemente prohibido y es motivo de descalificación.

En paralelo cada grupo realizará una breve **presentación** donde se explicarán las modificaciones introducidas para mejorar el comportamiento en simulación respecto a la Práctica 4 y, sobre todo, se hará especial hincapié en los cambios realizados para portar el código al robot real. Una vez concluidas la simulación y las presentaciones, se procederá a la competición con los robots físicos sobre un entorno idéntico al de la simulación —salvo pequeños errores de montaje— . En esta ocasión, los equipos competirán secuencialmente. Primero todos los equipos realizarán un intento y, a continuación, se repetirá la ronda de nuevo. De forma análoga a la simulación, solo se contabilizará el menor tiempo de ejecución.

Se concederán **puntos extra** a aquellos que equipos destaquen en ambas fases (Tabla 1). Además, se premiará con 0,5 puntos adicionales al equipo que consiga el menor tiempo combinado. Cada clase (A y B) competirá de manera independiente.

Tabla 1. Puntos extra por clase para los mejores clasificados en cada una de las fases.

Categoría	1er puesto	2° puesto	3er puesto
Simulación	+0,75 puntos	+0,50 puntos	+0,25 puntos
Robot físico	+0,75 puntos	+0,50 puntos	+0,25 puntos

Sugerencias

Dado que todos sus competidores empiezan básicamente con los mismos módulos que usted, necesita destacar sobre la multitud de alguna forma. A continuación puede encontrar una lista no exhaustiva de ideas que quizá quiera considerar.

Exploración

- Cuanto más rápido se mueva su robot, más información será capaz de adquirir para la localización y menos tiempo tardará en recorrer la ruta posteriormente, de modo que **mejorar sus algoritmos de control** (tanto el de exploración como el de seguimiento) definitivamente merece la pena. Tampoco conviene subir la velocidad lineal alegremente y dejar poco el margen para el giro. Es mejor tardar un poco más en llegar, que chocarse y no hacerlo nunca.

Localización

- Intente optimizar los **hiperparámetros** del filtro de partículas: los ruidos de movimiento y medida, el subconjunto de haces del LiDAR que utiliza...
- **No tiene mucho sentido remuestrear continuamente** porque la información recogida por los sensores será prácticamente idéntica. La implementación que tiene de la Práctica 3 remuestrea cada 10 periodos en simulación —y seguramente más en el robot real—, pero se trata de una elección arbitraria. Posiblemente convenga ajustar este valor, pero tenga en cuenta que depende de la velocidad de exploración. Si el robot se mueve rápido y remuestrea cada mucho tiempo, puede pasar por alto características relevantes del entorno. Para reducir la carga computacional, quizá quiera registrar la odometría de todos los periodos, integrarla para determinar las velocidades lineal y angular equivalentes y ejecutar las actualizaciones de movimiento y remuestreo solo una vez en lugar de ejecutar la predicción tantas veces como datos de odometría tenga almacenados.
- Para reducir el tiempo de ejecución del filtro, a medida que disminuye el número de hipótesis, se debe **adaptar el número de partículas** que se remuestrean. Acertar con la política de reducción es crítico. Si es demasiado agresiva, puede que el robot nunca se localice porque desaparezcan todas las partículas coherentes con la hipótesis correcta durante un remuestreo. Por el contrario, si es demasiado conservador, el filtro tardará un tiempo desesperadamente largo en converger. Puede intentar diseñar esta política usted mismo, o implementar la propuesta del filtro de partículas adaptativo o AMCL (del inglés, *Adaptive Monte Carlo Localization*), que utiliza la divergencia de Kullback-Leibler para determinar cuantas partículas hacen falta para representar fielmente la distribución de probabilidad subyacente[3].
- Eventualmente, el filtro de partículas puede **converger a una ubicación incorrecta**. Ser capaz de detectar esta situación puede sacarle de un apuro. Cuando considere que se ha localizado, compruebe la verosimilitud media de las medidas de los sensores con las que se obtienen cuando el robot converge a la pose correcta. Si hay demasiada discrepancia, el robot está perdido. La solución consiste en reiniciar el filtro por completo o generar nuevas partículas en las ubicaciones que puede haber descartado prematuramente.

[3] D. Fox, "KLD-sampling: Adaptive particle filter", en *Proc. 15th Int. Conf. Neural Information Processing Systems: Natural and Synthetic*, Vancouver, BC, Canadá, 3–8 Dic. 2001. URL: https://papers.nips.cc/paper_files/paper/2001/file/c5b2cebf15b205503560c4e8e6d1ea78-Paper.pdf

- Una vez que el robot se ha localizado, es posible sustituir el filtro de partículas por un **filtro de Kalman extendido (EKF)** para realizar el seguimiento de la pose de forma más eficiente computacionalmente. Si decide optar por este enfoque, es probable que surja una dificultad importante: ¿cómo calcular el Jacobiano del modelo de observación si únicamente se utilizan las distancias medidas por varios haces del LiDAR, sin extraer características geométricas (como líneas o ángulos), tal y como se hace en el EKF visto en clase?

  En este caso, el modelo de observación no tiene una expresión analítica sencilla. Por ello, una solución práctica consiste en aproximar el Jacobiano mediante **diferencias finitas**. La idea es la siguiente: para cada componente del vector de estado (posición y orientación), se introduce una pequeña perturbación y se observa cómo cambian las distancias medidas por el LiDAR. Las derivadas parciales necesarias para el Jacobiano se aproximan numéricamente a partir de esas variaciones.

- En lugar de moverse de manera aleatoria por el entorno hasta localizarse, puede **optimizar su estrategia de exploración** para descartar cuanto antes hipótesis incorrectas en zonas ambiguas[4]. Básicamente, en cada intersección debería escoger el camino que se espera que proporcione más información. Este algoritmo tiene un nombre, POMDP las siglas en inglés de "proceso de decisión de Markov parcialmente observable" (*Partially Observable Markov Decision Process*). Este problema es parcialmente observable porque el robot no está seguro de cuál es su pose, de modo que todas sus decisiones deberían estar ponderadas por sus creencias...

- **Vectorizar el código de Python** o programar algunos métodos —como se ha hecho con el cálculo de las intersecciones— o el nodo completo en C++ también pueden hacer que los tiempos de computación se reduzcan sustancialmente[5]. La única pega de reprogramar el nodo completo es que deberá portar también algunas funcionalidades de la clase `Map`. Como no la ha programado usted, no dude en pedir ayuda a los profesores si lo necesita. Los modelos extensos de lenguaje (LLM) también deberían ayudar bastante a realizar esta traducción. Son especialmente potentes si se les proporciona una implementación correcta y optimizada en otro lenguaje.

Planificación

- Debe decidir si utiliza la **inicialización** probabilística estándar o la de rejilla para el PRM y cuantos nodos generar.

- Añadir suficientes puntos adicionales a la ruta una vez localizada puede ayudar a que la ruta suavizada sea más fácil de seguir. Asegúrese de controlar con los pesos de los datos y el suavizado, o de alguna otra forma, que la ruta **no invade la zona de exclusión** o el robot podría chocarse al seguirla.

- También puede probar **otros algoritmos de planificación**, como el RRT* inverso, que precalcula la ruta al objetivo desde cualquier punto del mapa. De esta forma, una vez se localice, ya sabrá qué camino debe seguir. No obstante, implementar esta solución no es tan sencillo —en especial la fase de reconexión— y puede ser computacionalmente exigente si no se hace bien.

- El entorno real nunca coincide perfectamente con el mapa. Por ello, seguir estrictamente la ruta planificada en condiciones ideales puede provocar colisiones. Además, aunque en el contexto del proyecto no sucederá, pueden aparecer obstáculos no contemplados inicialmente, tanto estáticos

[4] A. R. Cassandra, L. P. Kaelbling y M. L. Littman, "Acting optimally in partially observable stochastic domains", en *Proc. 12th Nat. Conf. Artificial Intelligence*, Ago. 1994. URL: https://www.cassandra.org/arc/papers/aaai94.pdf

[5] Actualmente, la librería `nanobind` (https://nanobind.readthedocs.io/en/latest/) es posiblemente la mejor alternativa para llamar a código programado en C++ desde Python. También puede considerar un librería algo más antigua denominada `pybind11` (https://pybind11.readthedocs.io/en/stable/basics.html).

como dinámicos. Para abordar este problema, generalmente se introduce un **planificador local (o de corto plazo)**, que complementa al planificador global (de largo plazo) que ya ha implementado. Este planificador local utiliza la información de los sensores en tiempo real para ajustar la ruta y evitar obstáculos. Una vez superado el obstáculo, el robot puede retomar la ruta global original o, si es necesario, replanificar una nueva desde su pose actual.

Seguimiento de la ruta

- La persecución pura funciona mejor con errores de ángulo pequeños. En el punto inicial puede que el robot esté orientado en sentido contrario al de la ruta que tiene que seguir. Si no lo ha hecho ya, aprovechando que el robot tiene cinemática diferencial, es buena idea girar en el sitio hasta que el robot esté más o menos **alineado con la ruta** antes de activar la persecución pura.
- En caso de que no lo haya analizado con suficiente detalle en la Práctica 4, procure ajustar la velocidad lineal y la distancia de anticipación en función del error de ángulo. Quizá pueda sacar alguna idea de la **persecución pura regulada**[6].
- También puede probar otros algoritmos de seguimiento de rutas —computacionalmente ligeros, porque si no la Raspberry Pi 4 del TurtleBot 3 no será capaz de ejecutarlos en tiempo real— que no tengan error en régimen permanente. ¿Se podría incorporar un término integral a la persecución pura?

[6] S. Macenski et al., "Regulated pure pursuit for robot path tracking", *Autonomous Robots*, vol. 47, págs. 685-–694, Jun. 2023. URL: https://arxiv.org/abs/2305.20026